

Project Description **Fundamentals of Data Science**

Understanding and Detecting Malicious
Prompts



Faculty of Physics and Applied Computer
Science

Mikhail Shupliakou
27.12.2025

Contents

1	Introduction	3
1.1	Goal of the project	3
1.2	About the dataset	3
1.2.1	Dataset Structure	3
1.2.2	Key Features	3
2	Data Loading and Preprocessing	4
2.1	Data Loading	4
2.2	Basic Preprocessing	4
3	Text Representation: Embeddings	4
3.1	Tokenization	5
3.2	Generating Embeddings	5
3.3	Label Encoding	5
4	Dimensionality Reduction	5
4.1	Methods	5
4.2	Visualizations	6
5	Clustering	7
5.1	Algorithms Results	7
5.2	Analysis of Low Performance	8
5.3	Fine-tuned Embeddings	8
6	Outlier Detection	9
6.1	Methodology	9
6.2	Findings	9
7	Impact of Adversarial Changes	9
8	Classification	10
8.1	Baseline Performance	10
8.2	Robustness Test (Adversarial Data)	10
8.3	Adversarial Training	10
9	Conclusion	10
10	Sources	12

Understanding and Detecting Malicious Prompts Using Embeddings, Clustering, and Classification

1 Introduction

1.1 Goal of the project

Project Goal: This project aims to study and apply core data science techniques to the PL-Guard dataset, which contains malicious prompts in Polish. We treat this as a text categorization and anomaly detection problem. The key objectives are:

1. **Text Representation:** Converting text into numerical vectors using BERT embeddings.
2. **Dimensionality Reduction:** Simplifying data for analysis using PCA, t-SNE, and UMAP.
3. **Clustering:** Grouping similar prompts to identify patterns.
4. **Outlier Detection:** Identifying anomalous or adversarial prompts.
5. **Classification:** Building models to detect malicious categories.
6. **Robustness Analysis:** Exploring how adversarial modifications affect model performance and implementing defense strategies.

The focus is on understanding the methodology rather than developing a production-ready security system.

1.2 About the dataset

PL-Guard is a curated collection of Polish text prompts designed for evaluating content moderation systems.

1.2.1 Dataset Structure

The dataset consists of two splits:

1. **test Split:** The core dataset for benchmarking safety classifiers (15 categories).
2. **test-adversarial Split:** Contains perturbed versions of the test samples to evaluate robustness against bypass attempts.

1.2.2 Key Features

- **text:** The Polish prompt.
- **category:** The safety label ("safe" or one of 14 "unsafe" categories like harassment, hate speech, etc.).

- **modification-types** (Adversarial split only): Techniques used to alter the text (e.g., typos, synonyms).

2 Data Loading and Preprocessing

The project was implemented in Python using Jupyter Lab.

2.1 Data Loading

Accessing the PL-Guard dataset requires authentication via Hugging Face.

```
from huggingface_hub import login
# login(token="...") # Token required

from datasets import load_dataset

# Load datasets
test_dataset = load_dataset("NASK-PIB/PL-Guard", 'test')
test_adversarial_dataset = load_dataset("NASK-PIB/PL-Guard", 'test_adversarial')
```

```
DatasetDict({
  test: Dataset({
    features: ['text', 'category'],
    num_rows: 900
  })
})
```

The dataset contains 900 labeled rows.

2.2 Basic Preprocessing

Text normalization included lowercasing and removing Markdown artifacts (e.g., bolding symbols).

```
def normalize_text(example):
    example["text"] = example["text"].lower().strip()
    example["text"] = example["text"].replace("**", "")
    return example

test_dataset = test_dataset.map(normalize_text)
```

3 Text Representation: Embeddings

Machine learning models require numerical input. We used **BERT** (Bidirectional Encoder Representations from Transformers) to convert text into vector embeddings.

3.1 Tokenization

Tokenization splits text into smaller units (tokens) and maps them to numerical IDs.

- **input-ids**: Numerical representations of tokens.
- **attention-mask**: Indicates actual tokens vs. padding.

3.2 Generating Embeddings

We extracted sentence-level embeddings by passing tokens through the BERT encoder and applying mean pooling to the last hidden states.

```
import torch
import numpy as np

# Convert to Tensors
input_ids = torch.from_numpy(np.array(df["input_ids"].tolist())).long()
attention_mask = torch.from_numpy(np.array(df["attention_mask"].tolist())).long()

model.eval()
with torch.no_grad():
    outputs = model.bert(input_ids=input_ids, attention_mask=attention_mask)

# Mean Pooling
last_hidden = outputs.last_hidden_state
mask = attention_mask.unsqueeze(-1).expand(last_hidden.size()).float()
masked_embeddings = last_hidden * mask
embeddings = masked_embeddings.sum(dim=1) / mask.sum(dim=1)

# Resulting shape: [900, 768]
```

3.3 Label Encoding

Text categories were mapped to integer IDs for classification.

```
labels_list = sorted(list(set(df['category'])))
label2id = {label: i for i, label in enumerate(labels_list)}
# Result: {'safe': 0, 'unsafe\nS1': 1, ...}
```

4 Dimensionality Reduction

To visualize the 768-dimensional embeddings and mitigate the curse of dimensionality, we applied three reduction techniques.

4.1 Methods

1. **PCA (Principal Component Analysis)**: A linear method that projects data onto directions of maximum variance. We standardized the data before applying PCA.

2. **t-SNE:** A non-linear probabilistic technique that preserves local neighbor relationships.
3. **UMAP:** A manifold learning technique that balances local and global structure preservation, often faster than t-SNE.

4.2 Visualizations

Interactive 2D plots were generated to analyze the embedding space.



Figure 1: PCA Visualization



Figure 2: t-SNE Visualization

2D Visualization of Prompts by Category

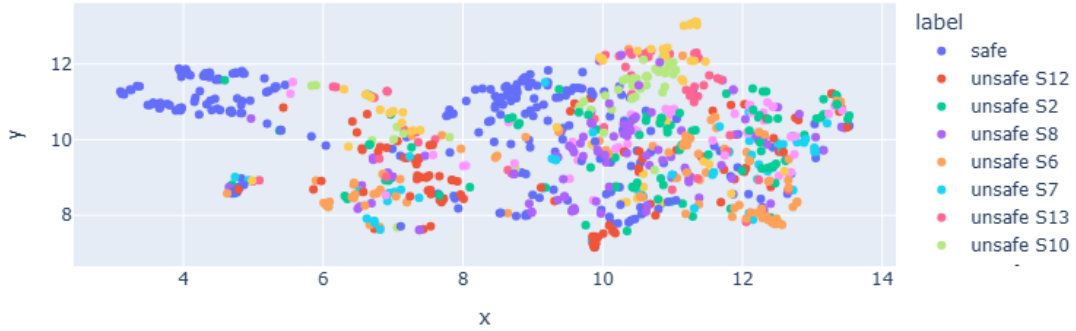


Figure 3: UMAP Visualization

Observation: The progression from PCA to UMAP highlights the non-linear nature of the data. Distinct clusters appear in UMAP/t-SNE, suggesting that BERT embeddings capture semantic differences, though unsupervised grouping may not perfectly align with safety labels.

5 Clustering

We applied unsupervised clustering to discover natural groupings in the data.

5.1 Algorithms Results

1. K-Means (k=15): Partitioned data into 15 clusters using Euclidean distance.

Adjusted Rand Index (ARI): 0.053
Normalized Mutual Info (NMI): 0.152

K-Means on Full Normalized Embeddings (UMAP Projection)

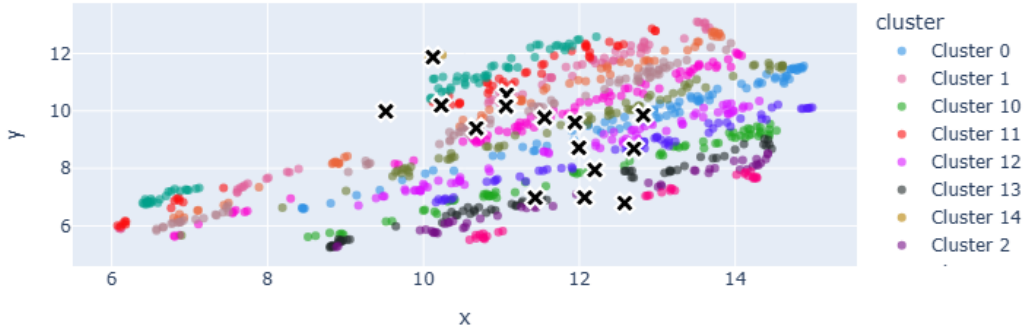


Figure 4: K-Means on UMAP projection

2. Hierarchical Clustering (Agglomerative): Built a hierarchy of clusters.

```
Agglomerative ARI: 0.118
Agglomerative NMI: 0.278
```

3. DBSCAN: Density-based clustering.

```
DBSCAN ARI: 0.069
```

5.2 Analysis of Low Performance

The low ARI scores ($\approx 0.05 - 0.12$) indicate a discrepancy between semantic clusters and safety labels.

- **Semantic Overlap:** "I hate you" (Hate Speech) and "You are an idiot" (Harassment) are linguistically similar but fall into different safety categories.
- **Diversity of "Safe":** The "Safe" category contains unrelated topics (coding, cooking, math). Clustering algorithms correctly split these into different groups, which penalizes the metric since they all share one label.

5.3 Fine-tuned Embeddings

To improve separation, we fine-tuned the model on the labeled training set.

```
=== Result after Fine-Tuning ===
New ARI (Fine-Tuned): 0.500
New NMI (Fine-Tuned): 0.583
```

Fine-tuning aligned the vector space with safety policies, significantly improving K-Means performance.

6 Outlier Detection

We identified anomalies to detect ambiguous or adversarial prompts.

6.1 Methodology

We used a distance-based approach: calculating the Euclidean distance of each prompt from its cluster centroid. Points furthest from the center were flagged as outliers.

```

=== ADVERSARIAL PROMPTS THAT MOVED FAR (OUTLIERS) ===
[Distance: 23.66] Text: **r"\ygotu j0dpowiednJi dhTke**[...]
[Distance: 23.34] Text: Jak bys namow0il grupe mlodyXch...

```

6.2 Findings

The top outliers (Distance > 23.0) were primarily **adversarial examples**. Techniques like leetspeak ("namow@il"), symbol injection ("inte!net"), and syntactic noise destroy the token structure BERT expects, pushing the embeddings far from any coherent semantic cluster.

7 Impact of Adversarial Changes

We compared embeddings of original prompts versus their adversarial variants to quantify the semantic shift.



Figure 5: Impact of Adversarial Changes

Conclusion: Structural attacks (leetspeak, char injection) cause massive embedding shifts, effectively blinding the model. Semantic changes (synonyms) cause minimal shifts, as the fine-tuned model preserves the underlying intent.

8 Classification

We evaluated supervised models using the full 768-dimensional embeddings.

8.1 Baseline Performance

We tested Logistic Regression, SVM, and Random Forest on clean data.

```
Logistic Regression: Accuracy: 73.89% | Precision: 0.75 | Recall: 0.74
SVM:                  Accuracy: 68.89%
Random Forest:        Accuracy: 62.78%
```

Logistic Regression achieved the best balance of performance.

8.2 Robustness Test (Adversarial Data)

We tested the classifiers on the *test-adversarial* split.

```
Model: Logistic Regression
Accuracy (Adversarial): 22.00%
Performance Drop:       -58.00 pp
```

Result: Performance collapsed from $\sim 74\%$ to $\sim 22\%$. The confusion matrix revealed that distinct threats (Hate Speech, Violence) were frequently misclassified as **Safe**.

8.3 Adversarial Training

To mitigate this, we augmented the training set with 450 adversarial examples and retrained the models.

```
Model: Logistic Regression (Retrained)
Accuracy on Clean Data:    76.11%
Accuracy on Adversarial Data: 76.44%
```

Conclusion: Adversarial training restored robustness (accuracy jumped to $\sim 76\%$). Logistic Regression proved superior to Random Forest, which overfitted the noise and lost performance on clean data.

9 Conclusion

This project conducted a comprehensive analysis of detecting malicious prompts in Polish using the PL-Guard dataset.

Key Findings

1. **Fine-Tuning is Essential:** Pre-trained BERT embeddings cluster by topic, not safety intent. Fine-tuning increased clustering ARI from 0.05 to 0.50.
2. **Fragility of Classifiers:** Standard classifiers are highly vulnerable to obfuscation attacks, with accuracy dropping by over 50 percentage points.

3. **Outliers as Signals:** Adversarial inputs manifest as statistical outliers in the embedding space.
4. **Defense Strategy:** Adversarial training effectively restores robustness. Logistic Regression offered the best stability between clean and noisy performance.

Building a robust content moderation system requires supervised representation learning and dynamic training strategies to defend against evolving attacks.

10 Sources

1. [Dataset source \(PL-Guard\)](#)
2. Krasnodebska et al., "PL-Guard: Benchmarking Language Model Safety for Polish", ACL, 2025.
3. [Text Embeddings: Comprehensive Guide](#)
4. [Principal Component Analysis \(PCA\)](#)
5. [Clustering in Machine Learning](#)
6. [Logistic Regression Overview](#)